

Basic Deep Neural Network

Building a Neural Network from Scratch with NumPy

A standalone teaching guide for absolute beginners. No prior machine-learning experience assumed. Every concept is explained in plain language, and every line of the companion script `basic_deep_neural_network.py` is accounted for.

WHAT YOU WILL BE ABLE TO DO

By the end of this guide you will understand what a deep neural network is, how it makes a prediction, how it measures its own mistakes, and how it teaches itself to improve - and you will be able to run a complete working example on your own computer.

1. What Is a Neural Network?

A neural network is a mathematical function that learns. You show it many examples of inputs paired with the correct answers, and it gradually adjusts itself until it can produce the right answer on its own - even for inputs it has never seen.

It is loosely inspired by the brain. The basic unit is a **neuron**: it takes some numbers in, multiplies each by a **weight** (a measure of importance), adds them up along with a **bias** (a constant offset), and passes the result through an **activation function** that decides how strongly the neuron 'fires'.

Neurons are organised into **layers**. The first layer receives your raw data, the last layer produces the answer, and any layers in between are called **hidden layers**. A network is called **deep** simply when it has more than one hidden layer.

THE SHAPE OF OUR NETWORK

The network in this guide follows the pattern: INPUT → [LINEAR → ReLU] repeated for each hidden layer → LINEAR → SIGMOID → OUTPUT. Don't worry about the names yet - each one is explained in its own section below.

2. The Big Picture: How a Network Learns

Training a neural network is a loop of four repeating steps. The whole rest of this guide is just a detailed look at each step.

Step	Name	Plain-English meaning
1	Forward propagation	Push the input through the network to get a prediction.
2	Compute cost	Measure how wrong the prediction is, as a single number.
3	Backward propagation	Work out how each weight contributed to the error.
4	Update parameters	Nudge every weight a little to reduce the error.

Run those four steps once and the network improves a tiny bit. Run them thousands of times and the network becomes genuinely good at the task. That repeating loop is the heart of nearly all deep learning.

3. The Building Blocks, One at a Time

The companion script is organised into the same steps shown below. Each function is one building block; reading them in order is like watching the network being assembled.

3.1 Activation functions

An activation function adds the 'bend' that lets a network learn curved, complex patterns instead of only straight lines. We use two:

- **ReLU** (Rectified Linear Unit): keeps positive numbers as-is and turns negatives into zero. It is simple, fast, and the default choice for hidden layers.
- **Sigmoid**: squashes any number into a value between 0 and 1, which we can read as a probability. We use it on the final layer for yes/no answers.

3.2 Initialize the parameters

Before training, the network's weights are set to small **random** numbers and the biases to **zero**. Randomness matters: if every weight started identical, every neuron would learn exactly the same thing and the network could never specialise.

```
parameters['W' + str(l)] = np.random.randn(layer_dims[l],
                                             layer_dims[l-1]) * 0.01
parameters['b' + str(l)] = np.zeros((layer_dims[l], 1))
```

3.3 Forward propagation (making a prediction)

Each layer does two things. First the **linear** step combines its inputs:

```
Z = np.dot(W, A) + b
```

Then the **activation** step bends the result with ReLU or sigmoid. Chaining this across all layers turns the raw input X into a final prediction the script calls AL.

3.4 Compute the cost

The **cost** is one number summarising how wrong the network is across every example. We use the cross-entropy cost, which punishes confident mistakes heavily. Training means driving this number as low as possible.

```
cost = -1/m * np.sum(Y*np.log(AL) + (1-Y)*np.log(1-AL))
```

3.5 Backward propagation (learning from mistakes)

Backpropagation is forward propagation run in reverse. Using calculus (the chain rule) it answers, for every single weight and bias: 'If I nudge you slightly, how much does the total error change?' Those answers are the **gradients**, written dW and db .

GRADIENTS ARE JUST DIRECTIONS

A gradient is simply an arrow saying 'increase this weight to raise the error' or 'decrease it to lower the error', plus how steeply. The network uses these arrows to know which way to adjust every knob.

3.6 Update the parameters (gradient descent)

Finally we adjust the knobs using one beautifully simple rule:

```
new_value = old_value - learning_rate * gradient
```

The **learning rate** is the size of each step. Too large and the network overshoots and never settles; too small and learning takes forever. This single rule, applied to every weight and bias, is called **gradient descent** - imagine walking downhill into the lowest valley of the error.

4. How to Run the Code

Follow these steps exactly:

Step 1 - Install Python and NumPy

Make sure Python 3 is installed, then install the one library this script needs:

```
python -m pip install numpy
```

Step 2 - Run the script

Open a terminal (Command Prompt or PowerShell on Windows) in the folder containing the file, then run:

```
python basic_deep_neural_network.py
```

Step 3 - Read the output

There are no data files to download - the script invents its own small dataset with a hidden rule for the network to discover. You will see the cost printed every 100 iterations. It starts near 0.69 and falls toward zero as the network learns:

```
Cost after iteration    0: 0.693147  
Cost after iteration 2000: 0.543944  
Cost after iteration 2400: 0.067361
```

Final training accuracy on the toy data: 99.5%

WHY THE COST IS FLAT AT FIRST

Because the weights start very small, the network is sluggish for the first ~1,800 iterations, then learning accelerates sharply. This slow-start-then-snap pattern is completely normal and is itself a useful thing to observe.

5. Quick Glossary

Term	Meaning
Weight (W)	A number measuring how important an input is to a neuron.
Bias (b)	A constant offset added inside a neuron.
Layer	A group of neurons. Input, hidden, and output layers stack together.
Activation	The function (ReLU/sigmoid) that adds non-linear 'bend'.
Forward prop	Pushing data through the network to get a prediction.
Cost	A single number measuring total error; lower is better.
Backprop	Computing how each weight contributed to the error.
Gradient	The direction and steepness to adjust a weight.
Learning rate	How big each adjustment step is.
Gradient descent	Repeatedly stepping 'downhill' to reduce the cost.
Epoch / iteration	One full pass of the forward/backward/update loop.

6. Where to Go Next

Once this network makes sense, natural next steps are: trying the network on a real dataset (such as images of cats vs. non-cats), experimenting with the learning rate and the number of layers, and then graduating to a framework like TensorFlow or PyTorch that handles backpropagation for you automatically. Everything those frameworks do under the hood is exactly what you just built by hand.